

Unified Admin Approval System - Remove Redundancy

❌ Current Problem: Duplicate Endpoints

You have:

```
GET /users/pending-doctors    ← Only doctors
POST /users/approve-doctor/{id} ← Only doctors

GET /users/pending-users      ← All users (doctors + staff)
POST /users/approve-user/{id} ← All users (doctors + staff)
```

This is redundant! You're maintaining two separate systems for the same thing.

✅ Recommended Solution: One Unified System

Keep **ONLY** these endpoints:

```
python
GET /admin/pending-approvals    # Get all pending (doctors + staff)
POST /admin/approve/{user_id}   # Approve or reject any user
GET /admin/pending-approvals/stats # Dashboard statistics
```

🔧 Implementation: Unified Approval Endpoints

Endpoint 1: Get Pending Approvals (Unified)

```
python
```

```

# app/api/routes/admin.py

from fastapi import APIRouter, Depends, Query, HTTPException
from sqlalchemy.orm import Session
from typing import Literal
from uuid import UUID

router = APIRouter(prefix="/admin", tags=["🛡️ Admin | Approvals"])

@router.get("/pending-approvals", response_model=UsersPublic)
async def get_pending_approvals(
    role: Literal["doctor", "staff", "all"] = "all", # Filter by role
    skip: int = Query(0, ge=0),
    limit: int = Query(100, ge=1, le=1000),
    current_user: User = Depends(get_current_admin),
    db: Session = Depends(get_db)
):
    """
    🛡️ **Access:** ADMIN only

    **Authentication:** DoctorOAuth2

    Get all users pending admin approval (doctors and/or staff).

    **Query Parameters:**
    - `role`: Filter by role
      - `all` (default): Both doctors and staff
      - `doctor`: Only doctors
      - `staff`: Only staff
    - `skip`: Pagination offset (default: 0)
    - `limit`: Max results (default: 100, max: 1000)

    **Returns:**
    - List of users awaiting approval
    - Total count

    **User must have:**
    - `is_verified = True` (email confirmed)
    - `is_approved = False` (pending approval)
    - `role IN ('doctor', 'staff')` (not admins)
    """

    # Base query: verified users pending approval
    query = db.query(User).filter(
        User.is_verified == True, # Email verified
        User.is_approved == False # Approval pending
    )

```

```
)

# Apply role filter
if role == "all":
    # Both doctors and staff (exclude admins)
    query = query.filter(User.role.in_(["doctor", "staff"]))
else:
    # Specific role only
    query = query.filter(User.role == role)

# Order by join date (newest first)
query = query.order_by(User.created_at.desc())

# Get total count
total_count = query.count()

# Get paginated results
users = query.offset(skip).limit(limit).all()

return UsersPublic(
    data=users,
    count=total_count
)
```

Endpoint 2: Approve/Reject User (Unified)

```
python
```

```
from pydantic import BaseModel, Field
```

```
from datetime import datetime
```

```
class ApprovalRequest(BaseModel):
```

```
    """Request body for approval/rejection"""
```

```
    action: Literal["approve", "reject"]
```

```
    reason: str | None = Field(
```

```
        None,
```

```
        description="Rejection reason (required if action=reject)",
```

```
        max_length=500
```

```
)
```

```
class ApprovalResponse(BaseModel):
```

```
    """Response after approval/rejection"""
```

```
    success: bool
```

```
    message: str
```

```
    user: UserPublic
```

```
@router.post("/approve/{user_id}", response_model=ApprovalResponse)
```

```
async def approve_or_reject_user(
```

```
    user_id: UUID,
```

```
    request: ApprovalRequest,
```

```
    current_user: User = Depends(get_current_admin),
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """
```

```
    🛡️ **Access:** ADMIN only
```

```
    **Authentication:** DoctorOAuth2
```

```
    Approve or reject a user's signup application.
```

```
    **Path Parameters:**
```

```
    - `user_id`: UUID of the user to approve/reject
```

```
    **Request Body:**
```

```
    - `action`: "approve" or "reject"
```

```
    - `reason`: Rejection reason (optional for approve, recommended for reject)
```

```
    **Actions:**
```

```
    **APPROVE:**
```

```
    - Sets `is_approved = True`
```

```
    - Sets `is_active = True` (can now login)
```

- Records approval timestamp and admin
- Sends approval email to user
- Logs action in audit trail

****REJECT:****

- Sets `is\_approved = FALSE`
- Sets `is\_active = FALSE` (cannot login)
- Stores rejection reason
- Sends rejection email with reason
- Logs action in audit trail

****Restrictions:****

- Only works for doctors and staff (not admins)
- User must be email verified
- Cannot approve already-approved users (idempotent)

****Returns:****

- Success status
- Message describing action
- Updated user object

"""

Validate action and reason

if request.action == "reject" and not request.reason:

```
    raise HTTPException(
        status_code=400,
        detail="Rejection reason is required when rejecting a user"
    )
```

Get the user

user = db.query(User).filter(User.id == user_id).first()

if not user:

```
    raise HTTPException(
        status_code=404,
        detail="User not found"
    )
```

Validate user role (only doctors and staff can be approved)

if user.role not in ["doctor", "staff"]:

```
    raise HTTPException(
        status_code=400,
        detail=f'Cannot approve/reject users with role '{user.role}'. This endpoint is only for doctors and staff.'
    )
```

Check if email is verified

if not user.is_verified:

```

raise HTTPException(
    status_code=400,
    detail="Cannot approve a user whose email is not verified yet"
)

# Check if already approved (prevent duplicate approvals)
if user.is_approved and request.action == "approve":
    return ApprovalResponse(
        success=True,
        message=f"User '{user.full_name}' is already approved",
        user=user
    )

# Perform the action
if request.action == "approve":
    # APPROVE
    user.is_approved = True
    user.is_active = True
    user.approved_at = datetime.utcnow()
    user.approved_by = current_user.id
    user.rejection_reason = None # Clear any previous rejection

    db.commit()
    db.refresh(user)

# Send approval email
send_approval_email(user)

# Log audit trail
create_audit_log(
    db=db,
    action="user_approved",
    user_id=user.id,
    performed_by=current_user.id,
    details={
        "user_name": user.full_name,
        "user_role": user.role,
        "user_email": user.email
    }
)

message = f"✅ {user.role.capitalize()} '{user.full_name}' has been approved and can now login"

else:
    # REJECT
    user.is_approved = False
    user.is_active = False

```

```

user.rejection_reason = request.reason
user.approved_at = None
user.approved_by = None

db.commit()
db.refresh(user)

# Send rejection email
send_rejection_email(user, request.reason)

# Log audit trail
create_audit_log(
    db=db,
    action="user_rejected",
    user_id=user.id,
    performed_by=current_user.id,
    details={
        "user_name": user.full_name,
        "user_role": user.role,
        "user_email": user.email,
        "reason": request.reason
    }
)

message = f"❌ {user.role.capitalize()} '{user.full_name}' has been rejected"

return ApprovalResponse(
    success=True,
    message=message,
    user=user
)

```

Endpoint 3: Dashboard Statistics (Bonus)

```
python
```

```
from typing import Dict
```

```
class ApprovalStats(BaseModel):
```

```
    """Dashboard statistics"""
```

```
    total_pending: int
```

```
    pending_doctors: int
```

```
    pending_staff: int
```

```
    pending_unverified_email: int # Signed up but haven't verified email yet
```

```
    approved_today: int
```

```
    rejected_today: int
```

```
@router.get("/pending-approvals/stats", response_model=ApprovalStats)
```

```
async def get_approval_stats(
```

```
    current_user: User = Depends(get_current_admin),
```

```
    db: Session = Depends(get_db)
```

```
):
```

```
    """
```

```
    🛡️ **Access:** ADMIN only
```

```
    Get approval statistics for admin dashboard.
```

```
    **Returns:**
```

```
    - Total pending approvals
```

```
    - Breakdown by role (doctors vs staff)
```

```
    - Users awaiting email verification
```

```
    - Approvals/rejections today
```

```
    """
```

```
    today = datetime.utcnow().date()
```

```
    # Total pending (email verified, approval pending)
```

```
    total_pending = db.query(User).filter(
```

```
        User.is_verified == True,
```

```
        User.is_approved == False,
```

```
        User.role.in_(["doctor", "staff"])
```

```
    ).count()
```

```
    # Pending doctors
```

```
    pending_doctors = db.query(User).filter(
```

```
        User.is_verified == True,
```

```
        User.is_approved == False,
```

```
        User.role == "doctor"
```

```
    ).count()
```

```
    # Pending staff
```

```

pending_staff = db.query(User).filter(
    User.is_verified == True,
    User.is_approved == False,
    User.role == "staff"
).count()

# Pending email verification
pending_unverified = db.query(User).filter(
    User.is_verified == False,
    User.role.in_(["doctor", "staff"])
).count()

# Approved today
approved_today = db.query(User).filter(
    User.approved_at >= today,
    User.is_approved == True,
    User.role.in_(["doctor", "staff"])
).count()

# Rejected today (users with rejection_reason set today)
# Note: You'd need a rejected_at timestamp field for this
# For now, we'll estimate based on updated_at
rejected_today = db.query(User).filter(
    User.updated_at >= today,
    User.is_approved == False,
    User.rejection_reason.isnot(None),
    User.role.in_(["doctor", "staff"])
).count()

return ApprovalStats(
    total_pending=total_pending,
    pending_doctors=pending_doctors,
    pending_staff=pending_staff,
    pending_unverified_email=pending_unverified,
    approved_today=approved_today,
    rejected_today=rejected_today
)

```



What to Remove (Deprecated Endpoints)

python

```
# DELETE THESE - They're redundant
```

```
@router.get("/users/pending-doctors") # ❌ Remove
```

```
async def get_pending_doctors(...):  
    pass
```

```
@router.post("/users/approve-doctor/{doctor_id}") # ❌ Remove
```

```
async def approve_doctor(...):  
    pass
```

```
# Keep the generic ones or migrate to new naming
```

```
@router.get("/users/pending-users") # ⚠️ Deprecated, use /admin/pending-approvals
```

```
@router.post("/users/approve-user/{user_id}") # ⚠️ Deprecated, use /admin/approve/{user_id}
```



Database Schema (Add Missing Fields)

```
sql  
-- Add these fields if they don't exist  
ALTER TABLE "user"  
ADD COLUMN IF NOT EXISTS approved_at TIMESTAMP NULL,  
ADD COLUMN IF NOT EXISTS approved_by UUID NULL REFERENCES "user"(id),  
ADD COLUMN IF NOT EXISTS rejection_reason TEXT NULL,  
ADD COLUMN IF NOT EXISTS created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
ADD COLUMN IF NOT EXISTS updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP;  
  
-- Create index for faster queries  
CREATE INDEX IF NOT EXISTS idx_user_approval_status  
ON "user"(is_verified, is_approved, role)  
WHERE role IN ('doctor', 'staff');
```



Updated Frontend: Admin Dashboard

```
html
```

```
<!DOCTYPE html>
<html>
<head>
  <title>Approval Dashboard - Admin</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }

    body {
      font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Arial, sans-serif;
      background: #f5f7fa;
      padding: 20px;
    }

    .container {
      max-width: 1400px;
      margin: 0 auto;
    }

    .header {
      background: white;
      padding: 24px;
      border-radius: 8px;
      margin-bottom: 20px;
      box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    }

    h1 {
      color: #1a202c;
      margin-bottom: 8px;
    }

    .subtitle {
      color: #718096;
      font-size: 14px;
    }

    /* Statistics Cards */
    .stats-grid {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
      gap: 16px;
      margin-bottom: 24px;
    }

    .stat-card {
      background: white;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    }

    .stat-value {
      font-size: 32px;
      font-weight: bold;
```

```
    color: #1a202c;
    margin-bottom: 4px;
}
.stat-label {
    font-size: 14px;
    color: #718096;
}
.stat-card.pending { border-left: 4px solid #f59e0b; }
.stat-card.doctors { border-left: 4px solid #3b82f6; }
.stat-card.staff { border-left: 4px solid #8b5cf6; }
.stat-card.approved { border-left: 4px solid #10b981; }

/* Filter Tabs */
.filters {
    background: white;
    padding: 16px 24px;
    border-radius: 8px;
    margin-bottom: 20px;
    box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    display: flex;
    gap: 12px;
    align-items: center;
}
.filter-label {
    font-weight: 600;
    color: #4a5568;
    margin-right: 8px;
}
.tab {
    padding: 8px 16px;
    border: 2px solid #e2e8f0;
    background: white;
    color: #4a5568;
    cursor: pointer;
    border-radius: 6px;
    font-size: 14px;
    font-weight: 500;
    transition: all 0.2s;
}
.tab:hover {
    border-color: #cbd5e0;
    background: #f7fafc;
}
.tab.active {
    background: #3b82f6;
    color: white;
    border-color: #3b82f6;
}
```

```
}

/* Table */
.table-container {
  background: white;
  border-radius: 8px;
  box-shadow: 0 1px 3px rgba(0,0,0,0.1);
  overflow: hidden;
}
table {
  width: 100%;
  border-collapse: collapse;
}
th {
  background: #f7fafc;
  color: #4a5568;
  padding: 14px 16px;
  text-align: left;
  font-weight: 600;
  font-size: 13px;
  text-transform: uppercase;
  letter-spacing: 0.5px;
  border-bottom: 2px solid #e2e8f0;
}
td {
  padding: 16px;
  border-bottom: 1px solid #e2e8f0;
  color: #2d3748;
}
tr:hover {
  background: #f7fafc;
}
tr:last-child td {
  border-bottom: none;
}

/* Role Badge */
.role-badge {
  display: inline-block;
  padding: 4px 10px;
  border-radius: 12px;
  font-size: 12px;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}
.role-doctor {
```

```
background: #dbeafe;
color: #1e40af;
}
.role-staff {
background: #ede9fe;
color: #6d28d9;
}

/* Buttons */
.btn {
padding: 8px 16px;
border: none;
border-radius: 6px;
cursor: pointer;
font-size: 13px;
font-weight: 500;
margin-right: 6px;
transition: all 0.2s;
}
.btn-approve {
background: #10b981;
color: white;
}
.btn-approve:hover {
background: #059669;
transform: translateY(-1px);
box-shadow: 0 2px 4px rgba(16, 185, 129, 0.3);
}
.btn-reject {
background: #ef4444;
color: white;
}
.btn-reject:hover {
background: #dc2626;
transform: translateY(-1px);
box-shadow: 0 2px 4px rgba(239, 68, 68, 0.3);
}

/* Empty State */
.empty-state {
text-align: center;
padding: 60px 20px;
color: #718096;
}
.empty-icon {
font-size: 64px;
margin-bottom: 16px;
```

```
}

/* Loading */
.loading {
  text-align: center;
  padding: 40px;
  color: #718096;
}

.spinner {
  border: 3px solid #e2e8f0;
  border-top: 3px solid #3b82f6;
  border-radius: 50%;
  width: 40px;
  height: 40px;
  animation: spin 1s linear infinite;
  margin: 0 auto 16px;
}

@keyframes spin {
  0% { transform: rotate(0deg); }
  100% { transform: rotate(360deg); }
}

/* Modal */
.modal {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  bottom: 0;
  background: rgba(0,0,0,0.5);
  align-items: center;
  justify-content: center;
  z-index: 1000;
}

.modal.show {
  display: flex;
}

.modal-content {
  background: white;
  padding: 24px;
  border-radius: 8px;
  max-width: 500px;
  width: 90%;
}

.modal-title {
  font-size: 18px;
```

```
    font-weight: 600;
    margin-bottom: 16px;
  }
  textarea {
    width: 100%;
    padding: 10px;
    border: 1px solid #e2e8f0;
    border-radius: 6px;
    font-family: inherit;
    resize: vertical;
  }
  .modal-actions {
    display: flex;
    gap: 10px;
    margin-top: 16px;
    justify-content: flex-end;
  }
  .btn-cancel {
    background: #e2e8f0;
    color: #4a5568;
  }
  .btn-cancel:hover {
    background: #cbd5e0;
  }
</style>
</head>
<body>
  <div class="container">
    <!-- Header -->
    <div class="header">
      <h1><img alt="Stethoscope icon" data-bbox="141 633 158 648"/> User Approval Dashboard</h1>
      <p class="subtitle">Review and approve doctor & staff signups</p>
    </div>

    <!-- Statistics -->
    <div class="stats-grid">
      <div class="stat-card pending">
        <div class="stat-value" id="stat-total">0</div>
        <div class="stat-label">Total Pending</div>
      </div>
      <div class="stat-card doctors">
        <div class="stat-value" id="stat-doctors">0</div>
        <div class="stat-label">Pending Doctors</div>
      </div>
      <div class="stat-card staff">
        <div class="stat-value" id="stat-staff">0</div>
        <div class="stat-label">Pending Staff</div>
      </div>
    </div>
  </div>
</body>
</html>
```

```
</div>

<div class="stat-card approved">
  <div class="stat-value" id="stat-approved">0</div>
  <div class="stat-label">Approved Today</div>
</div>
</div>

<!-- Filters -->
<div class="filters">
  <span class="filter-label">Show:</span>
  <button class="tab active" data-filter="all">All Pending</button>
  <button class="tab" data-filter="doctor">Doctors</button>
  <button class="tab" data-filter="staff">Staff</button>
</div>

<!-- Table -->
<div class="table-container">
  <div id="loading" class="loading">
    <div class="spinner"></div>
    <div>Loading pending approvals...</div>
  </div>

  <table id="users-table" style="display: none;">
    <thead>
      <tr>
        <th>Role</th>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Details</th>
        <th>Joined</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody id="users-tbody">
    </tbody>
  </table>

  <div id="empty-state" class="empty-state" style="display: none;">
    <div class="empty-icon">✔</div>
    <div><strong>All caught up!</strong></div>
    <div>No pending approvals at the moment.</div>
  </div>
</div>
</div>

<!-- Rejection Modal -->
```

```

<div class="modal" id="reject-modal">
  <div class="modal-content">
    <h3 class="modal-title">Reject User Application</h3>
    <p style="margin-bottom: 16px; color: #718096;">
      Please provide a reason for rejection:
    </p>
    <textarea id="rejection-reason" rows="4"
      placeholder="e.g., Invalid medical license number, incomplete information..."></textarea>
    <div class="modal-actions">
      <button class="btn btn-cancel" onclick="closeRejectModal()">Cancel</button>
      <button class="btn btn-reject" onclick="confirmReject()">Reject User</button>
    </div>
  </div>
</div>

<script>
  const API_BASE = 'http://localhost:8000';
  const token = localStorage.getItem('admin_token');
  let currentFilter = 'all';
  let pendingRejectUserId = null;

  // Tab filtering
  document.querySelectorAll('.tab').forEach(tab => {
    tab.addEventListener('click', function() {
      document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
      this.classList.add('active');
      currentFilter = this.dataset.filter;
      loadPendingUsers();
    });
  });

  // Load statistics
  async function loadStats() {
    try {
      const response = await fetch(`${API_BASE}/admin/pending-approvals/stats`, {
        headers: { 'Authorization': `Bearer ${token}` }
      });

      if (!response.ok) throw new Error('Failed to fetch stats');

      const stats = await response.json();
      document.getElementById('stat-total').textContent = stats.total_pending;
      document.getElementById('stat-doctors').textContent = stats.pending_doctors;
      document.getElementById('stat-staff').textContent = stats.pending_staff;
      document.getElementById('stat-approved').textContent = stats.approved_today;

    } catch (error) {

```

```
        console.error('Error loading stats:', error);
    }
}

// Load pending users
async function loadPendingUsers() {
    const loading = document.getElementById('loading');
    const table = document.getElementById('users-table');
    const empty = document.getElementById('empty-state');

    loading.style.display = 'block';
    table.style.display = 'none';
    empty.style.display = 'none';

    try {
        const url = `${API_BASE}/admin/pending-approvals?role=${currentFilter}`;
        const response = await fetch(url, {
            headers: { 'Authorization': `Bearer ${token}` }
        });

        if (!response.ok) {
            if (response.status === 401 || response.status === 403) {
                alert('Please login as admin');
                window.location.href = '/login.html';
                return;
            }
            throw new Error('Failed to fetch');
        }

        const data = await response.json();

        loading.style.display = 'none';

        if (data.data.length === 0) {
            empty.style.display = 'block';
        } else {
            table.style.display = 'table';
            displayUsers(data.data);
        }
    } catch (error) {
        console.error('Error:', error);
        loading.style.display = 'none';
        alert('Failed to load pending users');
    }
}
```

```
// Display users in table
```

```
function displayUsers(users) {  
  const tbody = document.getElementById('users-tbody');  
  tbody.innerHTML = "";  
  
  users.forEach(user => {  
    const roleClass = `role-${user.role}`;  
    const roleBadge = `<span class="role-badge ${roleClass}">${user.role}</span>`;  
  
    // Build details column based on role  
    let details = "";  
    if (user.role === 'doctor') {  
      details = `  
        <div><strong>${user.specialization || 'N/A'}</strong></div>  
        <div style="font-size: 12px; color: #718096;">  
          Reg: ${user.registration_number || 'N/A'}  
        </div>  
        <div style="font-size: 12px; color: #718096;">  
          ${user.clinic_name || 'No clinic'}  
        </div>  
      `;  
    } else {  
      details = `<span style="color: #a0aec0;">Staff member</span>`;   
    }  
  
    const joinDate = new Date(user.join_date || user.created_at).toLocaleDateString('en-US', {  
      month: 'short',  
      day: 'numeric',  
      year: 'numeric'  
    });  
  
    const row = document.createElement('tr');  
    row.innerHTML = `  
      <td>${roleBadge}</td>  
      <td><strong>${user.full_name}</strong></td>  
      <td style="color: #718096;">${user.email}</td>  
      <td style="color: #718096;">${user.phone || 'N/A'}</td>  
      <td>${details}</td>  
      <td style="color: #718096;">${joinDate}</td>  
      <td style="white-space: nowrap;">  
        <button class="btn btn-approve" onclick="approveUser('${user.id}', '${user.full_name}')">  
          ✓ Approve  
        </button>  
        <button class="btn btn-reject" onclick="openRejectModal('${user.id}')">  
          ✗ Reject  
        </button>  
      </td>  
    `;  
    tbody.appendChild(row);  
  });  
}
```

```

    `;
    tbody.appendChild(row);
  });
}

// Approve user
async function approveUser(userId, userName) {
  if (!confirm(`Approve ${userName}?`)) return;

  try {
    const response = await fetch(`${API_BASE}/admin/approve/${userId}`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ action: 'approve' })
    });

    if (response.ok) {
      const result = await response.json();
      alert(result.message);
      loadPendingUsers();
      loadStats();
    } else {
      const error = await response.json();
      alert('Approval failed: ' + error.detail);
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Approval failed');
  }
}

// Open reject modal
function openRejectModal(userId) {
  pendingRejectUserId = userId;
  document.getElementById('rejection-reason').value = '';
  document.getElementById('reject-modal').classList.add('show');
}

// Close reject modal
function closeRejectModal() {
  pendingRejectUserId = null;
  document.getElementById('reject-modal').classList.remove('show');
}

```

```

// Confirm rejection
async function confirmReject() {
  const reason = document.getElementById('rejection-reason').value.trim();

  if (!reason) {
    alert('Please provide a rejection reason');
    return;
  }

  try {
    const response = await fetch(`${API_BASE}/admin/approve/${pendingRejectUserId}`, {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        action: 'reject',
        reason: reason
      })
    });

    if (response.ok) {
      const result = await response.json();
      alert(result.message);
      closeRejectModal();
      loadPendingUsers();
      loadStats();
    } else {
      const error = await response.json();
      alert('Rejection failed: ' + error.detail);
    }
  } catch (error) {
    console.error('Error:', error);
    alert('Rejection failed');
  }
}

// Close modal on outside click
document.getElementById('reject-modal').addEventListener('click', function(e) {
  if (e.target === this) {
    closeRejectModal();
  }
});

```

```
// Initial load
loadStats();
loadPendingUsers();

// Auto-refresh every 30 seconds
setInterval(() => {
  loadStats();
  loadPendingUsers();
}, 30000);
</script>
</body>
</html>
```

Usage Examples

Example 1: Get All Pending Users

```
bash
# Get all pending (doctors + staff)
GET /admin/pending-approvals?role=all

# Get only pending doctors
GET /admin/pending-approvals?role=doctor

# Get only pending staff
GET /admin/pending-approvals?role=staff

# With pagination
GET /admin/pending-approvals?role=all&skip=0&limit=20
```

Example 2: Approve a User

```
bash
POST /admin/approve/550e8400-e29b-41d4-a716-446655440000

Request Body:
{
  "action": "approve"
}

Response:
{
  "success": true,
  "message": "✔ Doctor 'Dr. Ahmed Khan' has been approved and can now login",
  "user": { ... }
}
```

Example 3: Reject a User

```
bash
POST /admin/approve/550e8400-e29b-41d4-a716-446655440000
```

Request Body:

```
{
  "action": "reject",
  "reason": "Invalid medical license number provided"
}
```

Response:

```
{
  "success": true,
  "message": "❌ Doctor 'Dr. Ahmed Khan' has been rejected",
  "user": { ... }
}
```

Example 4: Get Statistics

```
bash
GET /admin/pending-approvals/stats
```

Response:

```
{
  "total_pending": 15,
  "pending_doctors": 12,
  "pending_staff": 3,
  "pending_unverified_email": 5,
  "approved_today": 8,
  "rejected_today": 2
}
```

✅ Summary

What We Fixed:

1. ❌ Removed Duplication

- Deleted `/users/pending-doctors` and `/users/approve-doctor/{id}`
- Unified into single endpoints that handle both roles

2. ✅ New Unified Endpoints

- `GET /admin/pending-approvals` - Get pending users (filterable by role)
- `POST /admin/approve/{user_id}` - Approve or reject any user

- `GET /admin/pending-approvals/stats` - Dashboard statistics

3. Improved Features

- Single approval workflow for both doctors and staff
- Better error messages
- Audit trail logging
- Statistics dashboard
- Rejection reasons required
- Idempotent approvals

4. Better Frontend

- Modern, professional UI
- Real-time statistics
- Role filtering
- Rejection modal with reason
- Auto-refresh

Migration Path:

```
python
# Old (DEPRECATED - Remove these)
GET /users/pending-doctors
POST /users/approve-doctor/{doctor_id}
GET /users/pending-users
POST /users/approve-user/{user_id}

# New (USE THESE)
GET /admin/pending-approvals?role=all|doctor|staff
POST /admin/approve/{user_id}
GET /admin/pending-approvals/stats
```

Result: Cleaner codebase, easier to maintain, one approval system for all user types!